# ShopLite: from a click to an order

DAY 1 / Understand the screen and the cart

## SEE

Use a small online shop.

Follow one customer journey.

## EXPLAIN

Connect JavaScript basics

to React, APIs and data.

## TRY

Make one small change.

Explain what happened.

**Our goal is to understand a working app, not type every file.**

# First, be the customer

LIVE DEMO / Explain the experience before the code

| 01 | 02 | 03 | 04 |
| --- | --- | --- | --- |
| **BROWSE** | **CART** | **ACCOUNT** | **ORDER** |
| Search and filter | Add two items | Register or log in | Place a demo order |

**This is a learning shop. No payment is collected.**

# Think of a small neighbourhood shop

ONE ANALOGY / Three different responsibilities

### BROWSER = COUNTER

Displays products.

Collects clicks and form input.

### SERVER = SHOPKEEPER

Checks requests.

Decides what is allowed.

### DATABASE = RECORDS

Keeps products, accounts and completed orders.

**The browser talks to the server. The server talks to MongoDB.**

# Give each tool one simple job

PROJECT MAP / Start with the two top-level folders

### client

React: screen components

CSS: appearance

Vite: development and build

### server

Node.js: runs JavaScript

Express: HTTP routes

Mongoose: models and queries

### MongoDB

Stores documents

Runs outside these folders

Keeps data across requests

**React is a library. Node.js is a runtime. MongoDB is a database.**

# A component is a reusable screen piece

REACT / Connect functions, inputs and JSX

Shortened from ProductCard.jsx

```
function ProductCard({ product, onAddToCart }) {
  return (
    <button onClick={() =>
onAddToCart(product._id)}>
      Add to cart
    </button>
  );
}
```

- Component: a function that returns UI.
- Props: inputs passed from a parent.
- JSX: HTML-like syntax inside JavaScript.
- A callback tells the parent what happened.

**Data goes down through props; callbacks carry user actions back up.**

# Five-minute pause

PAUSE / Leave the browser and editor ready

**A component is a reusable piece.**
**Props are its inputs.**

When we return: what makes the screen change?

**Take a short break. No new concept here.**

# State is the screen's changing memory

REACT / Click -> update state -> render again

Excerpts from App.jsx

```
const [search, setSearch] = useState("");

<input
  value={search}
  onChange={(event) =>
    setSearch(event.target.value)}
/>
```

- search is the current value.
- setSearch requests a state update.
- React renders the updated UI.
- A normal variable is not reactive state.

**The displayed UI follows the current state.**

# Cart memory is not an order

BROWSER DATA / Understand what survives a refresh

### STATE

Current cart in React.

Each line has productId and quantity.

### localStorage

Save cart in this browser.

Key: shoplite_cart

Survives a refresh.

### DATABASE ORDER

Created only at checkout.

Belongs to a user.

Stored by the server.

**A cart is a browser-side selection. An order is a saved server record.**

# How do products reach the screen?

DATA FLOW / A first look at useEffect

**01**

**APP LOADS**

Effect requests products

**02**

**API**

GET /api/products

**03**

**STATE**

Save data.products

**04**

**SCREEN**

Render ProductCard list

**An effect synchronizes React with something outside the render.**

# Try it: change one visible thing

PAIR LAB / 14 minutes, including a short share-back

### 1 CHANGE

In ProductCard.jsx, change the button text to Add to my bag.

### 2 CHECK

Refresh and click it. The cart should still work. Then restore the label.

### 3 EXPLAIN

Search for bottle. Identify the state, event and filtered result.

**Success means the label changes without breaking Add to cart.**

# Day 1: say it without jargon

EXIT TICKET / Answer before looking at the notes

1. What is a component? What are props?
2. What makes React update the screen?
3. Does adding to the cart call the order API?
4. Why does the cart survive a refresh?

**Tomorrow: follow the product request into the server.**

# Day 2: behind the screen

RECAP / Follow one product request all the way back

**The browser asks.**
**The server checks.**
**The database remembers.**

Today: HTTP, routes, models and registration.

**Keep the shop analogy. Add one technical word at a time.**

# An API is an agreed way to ask

HTTP / Method + path + optional body

### REQUEST

GET /api/products

"Send the product list."

Usually no request body.

### SERVER WORK

Choose the route.

Check the request.

Read or write data.

### RESPONSE

Status: was it successful?

JSON: the returned data

or an error message.

**GET reads data. POST submits data to create or process something.**

# The entire API fits on one slide

ROUTE MAP / Two order routes need sign-in

| Method | Path | Purpose | Sign-in? |
|---|---|---|---|
| GET | /api/health | API status | No |
| GET | /api/products | Active products | No |
| POST | /api/auth/register | Create account | No |
| POST | /api/auth/login | Sign in | No |
| POST | /api/orders | Create demo order | Yes |
| GET | /api/orders | My order history | Yes |

**A route is identified by both its HTTP method and its path.**

# JSON is just structured information

READ A RESPONSE / The shape matters

Illustrative response; extra product fields omitted

```
{
  "products": [
    {
      "_id": "<database-generated id>",
      "name": "Insulated Bottle",
      "price": 799,
      "category": "Lifestyle"
    }
  ]
}
```

- An object has named fields.
- An array holds multiple products.
- price is a number, not display text.
- React reads data.products.

**The response is { products: [...] }, not a bare array.**

# Three models describe the shop

MONGOOSE / A schema describes fields and rules

## PRODUCT

Name, category, price

Description, image, active

One catalogue item

## USER

Name and email

passwordHash

One customer account

## ORDER

User reference and items

Saved name, price, quantity

Total, status and timestamps

**A schema describes a document. A model lets code work with it.**

# Five-minute pause

PAUSE / Keep the API response open

## Request -> route -> model -> data

Next: trace the shortest real route in the project.

**Take a short break. No new concept here.**

# Read one route, line by line

PRODUCTS / A small route teaches the whole pattern

productRoutes.js, line-wrapped for teaching

```
router.get("/", async (req, res, next) => {
  try {
    const products = await Product
      .find({ active: true })
      .sort({ name: 1 })
      .lean();
    return res.json({ products });
  } catch (error) {
    return next(error);
  }
});
```

- get: respond to a read request.
- await: wait for the query result.
- find: only active products.
- res.json: send the response.

**Trace the code in the same order as the request.**

# Validation is the server saying "check first"

ERRORS / A friendly form is not the final authority

| Status | Meaning here | ShopLite example |
| --- | --- | --- |
| 200 / 201 | Success / created | Products / new account |
| 400 | Invalid input | Quantity outside 1-20 |
| 401 | Not authenticated | Missing or expired token |
| 404 | No matching route | Unknown API path |
| 409 | Conflict | Email already registered |
| 500 | Server error | Unexpected server failure |

**Validate on the server even when the form already checks input.**

# Registration: store a hash, not a password

ACCOUNT CREATION / Four understandable steps

| 01 | 02 | 03 | 04 |
| --- | --- | --- | --- |
| **VALIDATE** | **CHECK** | **HASH + SAVE** | **RESPOND** |
| Name, email, password | Email is not registered | bcrypt -> passwordHash | Token + public user |

**Hashing is not encryption. Login compares against the hash.**

# Try it: inspect, then predict

PAIR LAB / Use the browser Network panel

### 1 INSPECT

Reload the shop.

Find GET /api/products.

Read its JSON response.

### 2 PREDICT

Which response field holds the list?

Which file reads it?

### 3 VERIFY

Search for bottle.

Does typing send another product request?

**Success: identify the request, its response shape and its consumer.**

# Day 2: finish the sentence

EXIT TICKET / Use your own words

1. GET /api/products is handled by ...
2. The Product model is used to ...
3. The response has a products field because ...
4. A password hash is stored instead of ...

**Tomorrow: identify a user and save their order.**

# Day 3: turn a cart into an order

RECAP / Identity, rules and persistence

**Who is asking?**
**What are they buying?**
**What should we save?**

Today: login, protected routes and demo checkout.

**The most important rules belong on the server.**

# Login: prove that the password matches

AUTHENTICATION / Recognize an existing customer

**01**

**FORM**

Email + password

**02**

**LOOK UP**

Find user by email

**03**

**COMPARE**

bcrypt checks the hash

**04**

**SESSION**

Token + public user

**Login checks identity. It does not create a new account.**

# A JWT is a signed pass, not a secret box

SESSION / The browser carries proof on later requests

### ISSUED

Server signs a token.

sub identifies the user.

Expiry is 2 hours here.

### CARRIED

Client stores token + user.

apiRequest sends the token

on protected requests.

### VERIFIED

Server checks signature

and expiry before allowing

the order handler to run.

**A signature detects tampering. It does not encrypt the token contents.**

# A protected route has a gatekeeper

MIDDLEWARE / Authentication comes before order logic

**01**

**REQUEST**

GET or POST /api/orders

**02**

**requireAuth**

Check signed token

**03**

**USER ID**

Set req.userId

**04**

**HANDLER**

Run order logic

**No valid token -> 401. The handler does not run.**

# Checkout sends choices, not trusted prices

REQUEST BODY / The browser does not set the final total

POST /api/orders body; replace the illustrative ID

```
{
  "items": [
    {
      "productId": "<real product _id>",
      "quantity": 2
    }
  ]
}
```

- Send productId and quantity.
- Attach the session token.
- Server loads current DB prices.
- Server creates the saved order.

**The server uses req.userId and database prices, not browser claims.**

# Five-minute pause

PAUSE / Prepare the final demo

**Browser: choices.**
**Server: trusted calculation.**

Next: save the receipt and see only your orders.

**Take a short break. No new concept here.**

# Save a receipt that does not change later

ORDER DATA / Totals, snapshots and ownership

## CALCULATE

2 bottles x INR 799

+ 1 lamp x INR 1,499

= INR 3,097

## REMEMBER

Save name, price, quantity.

Save total and user ID.

Keep the order timestamp.

## SHOW MY ORDERS

Query user: req.userId.

Return newest orders first.

Do not return everyone's.

**Order items keep the name and price from checkout time.**

# Good screens explain waiting and failure

ASYNC UI / A request is not instantly successful

## WAITING

Loading products...

Please wait...

Placing order...

## SUCCESS

Show returned data.

Clear cart after checkout.

Open order history.

## FAILURE

Show an error message.

Keep cart after failure.

Let the user correct it.

**A disabled button reduces repeated clicks; it is not server idempotency.**

# Trace one complete order together

GUIDED DEMO / Narrate every boundary crossing

| 01 | 02 | 03 | 04 |
|---|---|---|---|
| **SELECT** | **SIGN IN** | **SUBMIT** | **READ BACK** |
| 2 bottles + 1 lamp | Identity is established | POST /api/orders | GET /api/orders |

**Use DevTools to connect what the user sees to what the API returns.**

# Try it: predict before you click

PAIR LAB / Explain three different outcomes

### SIGNED OUT

Add an item.

Click Place demo order.

Where does the UI go?

### SIGNED IN

Return to the cart.

Place the demo order.

Refresh; reopen My orders.

### DIFFERENT USER

Use another test account.

Open My orders.

Whose records appear?

**Success: explain identity, persistence and ownership separately.**

# Explain ShopLite in one minute

FINAL EXIT TICKET / The story matters more than memorizing code

1. Which layer displays the product cards?
2. Which layer checks the token and the prices?
3. Where are completed orders stored?
4. Why is the cart not the same as an order?

**React displays. Express checks. MongoDB stores.**

# If you have only two days

TEACHER APPENDIX / Two 90-minute sessions

## DAY A / 90 MIN

Demo + analogy: 15
React + cart: 25
Break: 5
Product API trace: 20
Guided lab: 20
Recap: 5

## DAY B / 90 MIN

Recap: 5
Login + JWT: 20
Checkout + ownership: 20
Break: 5
End-to-end demo/lab: 30
Final explanation: 10

## REDUCE THE SCOPE

Use a pre-running app.
Demo registration only.
Skip schema detail.
Move coding to homework.
Keep the final order trace.

**Two days is an introduction, not a from-scratch full-stack build.**

# Set up before students arrive

TEACHER APPENDIX / Keep installation out of the 90 minutes

## PREPARE

Node.js + npm; MongoDB

Follow README setup.

Use a disposable class DB.

Keep secrets off the screen.

## RUN

In server: npm install
npm run seed; npm run dev
In client: npm install
npm run dev
Use separate terminals.

## VERIFY

API: localhost:5000/api/health
Products: /api/products
UI: localhost:5173
Test login and demo order.
Keep a backup demo ready.

**PowerShell blocks npm.ps1? Use npm.cmd with the same arguments.**

# When the demo does not work

TEACHER APPENDIX / Diagnose one layer at a time

| Symptom | First check | Next step |
| --- | --- | --- |
| UI cannot load | Vite terminal and URL | Open its printed local URL |
| API unreachable | Backend terminal | Check startup and API URL |
| Products empty | GET /api/products | Check intended DB and seed |
| Request blocked | Origin and ports | Align CORS with client origin |
| Orders return 401 | Missing/expired session | Log out, then log in again |
| Product image missing | External image network | Use names; continue lesson |

**Read the terminal and Network panel before changing code.**

# A small vocabulary, a clear mental model

REFERENCE / Return to these meanings

| Term | Simple meaning | ShopLite example |
| --- | --- | --- |
| Component / props | UI piece / its inputs | ProductCard receives product |
| State / effect | UI memory / outside sync | Cart / product request |
| API / route | Request agreement / handler | GET /api/products |
| Schema / model | Data rules / data interface | Product schema / Product.find |
| Authentication | Checking identity | Login and token verification |
| Authorization | Checking allowed access | Only my order history |

**Next steps: stronger testing and production design, not real payments today.**